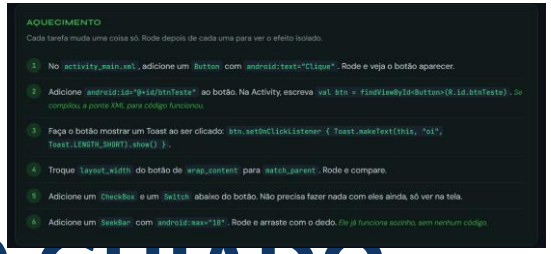
Universidade POSITIVO



DESENVOLVIMENTO MOBILE — PROJETO GUIADO

UP Campus+ • do primeiro clique ao aplicativo final

Como este material funciona

Você vai construir um único aplicativo do começo ao fim.

Não existe capítulo de teoria antes da prática.

Cada missão acrescenta uma funcionalidade ao UP Campus+.

Se aparecer um termo novo, consulte o mini-glossário do bloco.

Se aparecer uma dúvida, pare exatamente naquele ponto e resolva.

Guarde cada checkpoint: o próximo passo parte do que já funciona.

REGRA → faça → rode → veja → mude → rode de novo.

O app que você vai entregar

UP Campus+ — Agenda Acadêmica

Tela inicial.

Cadastro de eventos: nome, data, local e descrição.

Lista de eventos.

Detalhes de um evento.

Editar e excluir.

Favoritar.

Dados continuam após fechar o app.

Tela Sobre.

APK final + demonstração do funcionamento.

Ao final, não haverá 'exercícios soltos': haverá um aplicativo.

Nossa versão do laboratório

Android Studio Giraffe | 2022.3.1

Todos os caminhos de menu e telas deste roteiro foram pensados para o ambiente Giraffe.

Projeto principal: XML/Views + Kotlin, porque cobre diretamente o plano de ensino.

Room entra como evolução da persistência local.

Compose aparece no final como comparação moderna, sem quebrar o app principal.

Abra o Android Studio. A partir daqui, tudo acontece nele.

MISSÃO 01 — FAÇA O APP NASCER

Criar o projeto e ver UP Campus+ rodando pela primeira vez.

Tempo sugerido: 20–30 min • Objetivo: terminar com algo funcionando.

Glossário rápido

Não decore. Consulte quando precisar.

Project
arquivos do projeto

Run
compila e executa

Activity
tela/controlador Android

Emulador
celular virtual

XML
estrutura visual da tela

res/layout
pasta das telas XML

Kotlin
comportamento do app

MainActivity
primeira Activity

Use o glossário como cola rápida. Depois volte para a missão.

Universidade POSITIVO

Crie o projeto

File → New → New Project.

Escolha Empty Views Activity.

Name: UPCampusPlus.

Language: Kotlin.

Finish.

Espere o Gradle terminar antes de continuar.

Encontre a tela

Project → app → res → layout.

Abra activity_main.xml.

Use Code ou Split.

Você está olhando para a primeira tela do app.

Não altere nada ainda. Só encontre o arquivo.

Universidade POSITIVO

Encontre o Kotlin

Project → app → java/kotlin → seu pacote.

Abra MainActivity.kt.

Localize onCreate().

Localize setContentView(R.layout.activity_main).

XML e Kotlin já estão conectados.

Universidade POSITIVO

Primeiro Run

Escolha um emulador/dispositivo.

Clique Run ▶.

Espere o app abrir.

Se abriu sem erro, checkpoint concluído.

CHECKPOINT ✓ o projeto vazio executa.

MISSÃO 02 — CRIE A HOME

Transformar a tela vazia em uma Home com título e botão COMEÇAR.

Tempo sugerido: 30–40 min • Objetivo: terminar com algo funcionando.

Glossário rápido

Não decore. Consulte quando precisar.

LinearLayout
organiza em linha/coluna

match_parent
ocupa o espaço disponível

TextView
mostra texto

wrap_content
ocupa só o necessário

Button
recebe clique

dp
dimensão/espaçamento

id
nome único do componente

sp
tamanho de texto

Use o glossário como cola rápida. Depois volte para a missão.

Substitua a tela inicial

Cole o XML completo e rode.

ONDE COLAR → activity_main.xml → substitua o conteúdo atual

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="24dp"
android:gravity="center">

<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="UP Campus+"
android:textSize="30sp" />

<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Agenda Acadêmica"
android:textSize="18sp" />

<Button
android:id="@+id/btnComecar"
android:layout_width="match_parent"
android:layout_height="wrap_content"
```

Run ▶ → a Home precisa aparecer.

```
</LinearLayout>
```

Mexa na tela

Troque “Agenda Acadêmica” por outro subtítulo.

30sp → 36sp.

24dp → 40dp.

Rode depois de cada alteração.

Descubra o efeito antes de decorar o nome.

Dê identidade ao botão

O botão já tem id: btnComecar.

Esse ID será usado no Kotlin.

Não mude o ID nos próximos passos.

CHECKPOINT ✓ Home pronta + botão identificado.

MISSÃO 03 — FAÇA O BOTÃO RESPONDER

Clicar em COMEÇAR e receber uma resposta do aplicativo.

Tempo sugerido: 25–35 min • Objetivo: terminar com algo funcionando.

Glossário rápido

Não decore. Consulte quando precisar.

findViewById
encontra uma View pelo ID

R.id
IDs gerados a partir do XML

val
variável somente leitura

Button
tipo do componente

setOnClickListener
ação executada no clique

Toast
mensagem temporária

Use o glossário como cola rápida. Depois volte para a missão.

Conecte XML e Kotlin

Agora o botão deixa de ser só visual.

ONDE COLAR → MainActivity.kt → dentro de onCreate(), depois de setContentView(...)

```
val btnComecar = findViewById<Button>(R.id.btnComecar)

btnComecar.setOnClickListener {
        Toast.makeText(
                this,
        "UP Campus+ iniciado!",
          Toast.LENGTH_SHORT
            ).show()
             }
```

ANTES DE CONTINUAR ✓ importe android.widget.Button e android.widget.Toast se o Android Studio solicitar.

Se Button ou Toast ficarem vermelhos, Alt + Enter e mandem importar

Run ▶ → clique COMEÇAR → veja o Toast.

Desafio sem copiar

Mude o comportamento.

Troque a mensagem do Toast.

Troque LENGTH_SHORT por LENGTH_LONG.

Rode e compare.

Explique: qual linha encontra o botão? qual linha reage ao clique?

CHECKPOINT ✓ XML cria o botão; Kotlin controla o botão.

MISSÃO 04 — ABRA A AGENDA

Criar uma segunda tela e navegar da Home para a Agenda.

Tempo sugerido: 40–50 min • Objetivo: terminar com algo funcionando.

Glossário rápido

Não decore. Consulte quando precisar.

Intent
pedido para abrir/executar algo

explicit Intent
destino conhecido pelo app

Activity
uma tela do app

Manifest
declara componentes do aplicativo

startActivity
abre a Activity indicada

Use o glossário como cola rápida. Depois volte para a missão.

Crie a segunda Activity

A Agenda será uma nova tela.

Clique com botão direito no pacote Kotlin.

New → Activity → Empty Views Activity.

Activity Name: AgendaActivity.

Layout Name: activity_agenda.

Finish.

Confirme que AgendaActivity.kt e activity_agenda.xml foram criados.

Monte a Agenda

Por enquanto, título + botão.

ONDE COLAR → activity_agenda.xml → substitua o conteúdo

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Minha Agenda"
        android:textSize="28sp" />

    <Button
        android:id="@+id/btnNovoEvento"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="NOVO EVENTO"
        android:layout_marginTop="20dp" />

</LinearLayout>
```

Ainda não rode. Falta abrir essa tela.

Troque o Toast pela navegação

O botão COMEÇAR agora abre a Agenda.

ONDE COLAR → MainActivity.kt → substitua o setOnClickListener anterior

```
btnComecar.setOnClickListener {
        val intent = Intent(
                this,
        AgendaActivity::class.java
                )
        startActivity(intent)
                }
```

ANTES DE CONTINUAR ✓ AgendaActivity precisa existir. Importe android.content.Intent se necessário.

Run ▶ → COMEÇAR → Minha Agenda.

Universidade POSITIVO

MICROMISSÃO • BOTÕES

Mais botões na Agenda

Vamos praticar componentes visuais antes de entrar no cadastro.

ONDE COLAR → activity_agenda.xml → dentro do LinearLayout, abaixo de NOVO EVENTO.

Adicione os botões SOBRE e VOLTAR.

Por enquanto, SOBRE será apenas visual. VOLTAR vai funcionar no próximo passo.

```
<Button
    android:id="@+id/btnSobre"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="SOBRE" />

<Button
    android:id="@+id/btnVoltar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="VOLTAR" />
```

Run ▶ → a Agenda deve mostrar NOVO EVENTO, SOBRE e VOLTAR.

Universidade POSITIVO

MICROMISSÃO • BOTÕES

Faça o botão VOLTAR funcionar

Agora vamos fechar a Activity atual e retornar para a anterior.

ONDE COLAR → AgendaActivity.kt → dentro de onCreate(), depois de setContentView(...).

ANTES DE CONTINUAR ✓ btnVoltar precisa existir no activity_agenda.xml.

finish() fecha a Activity atual. Como a Agenda foi aberta pela Home, voltamos para a Home.

```
val btnVoltar = findViewById<Button>(R.id.btnVoltar)

btnVoltar.setOnClickListener {
    finish()
}
```

Run ▶ → COMEÇAR → Agenda → VOLTAR → Home.

Universidade POSITIVO

MICROMISSÃO • CHECKPOINT

Entrar e voltar

Use o que já construímos para entender a navegação.

COMEÇAR → usa Intent para abrir AgendaActivity.

VOLTAR → usa finish() para fechar AgendaActivity.

SOBRE → já está na tela, mas será conectado em uma missão futura.

DESAFIO → troque o texto de VOLTAR, rode e depois restaure.

CHECKPOINT ✓ Home abre Agenda e VOLTAR retorna sem travar.

MISSÃO 05 — CADASTRE UM EVENTO

Criar formulário com nome, data, local e descrição.

Tempo sugerido: 50–60 min • Objetivo: terminar com algo funcionando.

Universidade POSITIVO

Glossário rápido

Não decore. Consulte quando precisar.

EditText
entrada de texto

hint
dica dentro do campo

inputType
tipo de entrada

TextView
saída de texto

trim
remove espaços extras

isEmpty
verifica campo vazio

Use o glossário como cola rápida. Depois volte para a missão.

Crie CadastroEventoActivity

Uma tela só para cadastrar.

New → Activity → Empty Views Activity.

Activity Name: CadastroEventoActivity.

Layout: activity_cadastro_evento.

Finish.

A Agenda vai abrir essa tela no próximo passo.

Monte o formulário

Quatro entradas + Salvar.

ONDE COLAR → activity_cadastro_evento.xml → substitua o conteúdo

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/etNome"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nome do evento" />

    <EditText
        android:id="@+id/etData"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Data" />

    <EditText
        android:id="@+id/etLocal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Local" />

    <EditText
        android:id="@+id/etDescricao"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Descrição" />

    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="SALVAR" />
```

A tela já existe. Agora precisamos chegar até ela.

Abra o cadastro

Agenda → Novo Evento → Cadastro.

ONDE COLAR → AgendaActivity.kt → onCreate(), depois de setContentView(...)

```
val btnNovoEvento =
findViewById<Button>(R.id.btnNovoEvento)

btnNovoEvento.setOnClickListener {
startActivity(
Intent(this, CadastroEventoActivity::class.java)
)
}
```

ANTES DE CONTINUAR ✓ importe Button e Intent se necessário

Run ▶ → COMEÇAR → NOVO EVENTO.

Valide o primeiro campo

Ainda não salvamos no banco.

ONDE COLAR → CadastroEventoActivity.kt → onCreate(), depois de setContentView(...)

```
val etNome = findViewById<EditText>(R.id.etNome)
val etData = findViewById<EditText>(R.id.etData)
val etLocal = findViewById<EditText>(R.id.etLocal)
val etDescricao = findViewById<EditText>(R.id.etDescricao)
val btnSalvar = findViewById<Button>(R.id.btnSalvar)

btnSalvar.setOnClickListener {
val nome = etNome.text.toString().trim()

if (nome.isEmpty()) {
etNome.error = "Informe o nome"
} else {
Toast.makeText(this, "Evento pronto para salvar", Toast.LENGTH_SHORT).show()
}
}
```

Teste com nome vazio e com nome preenchido.